

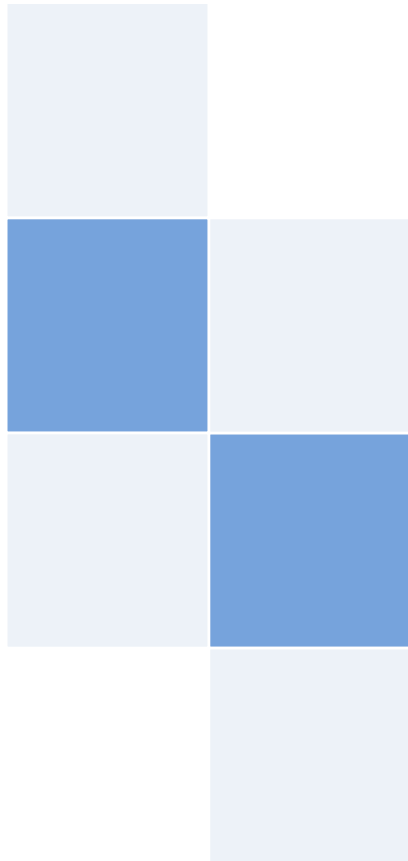
M10. Administració de Bases de Dades

UF1. Llenguatge SQL: DCL i extensió procedimental

Autor: Robert Ventura Vall-llovera
Data creació: Setembre 2015
Última actualització: Abril 2021



Aquest document està subjecte a una llicència de [Reconeixement-No comercial-Compartir Igual 3.0 Espanya](https://creativecommons.org/licenses/by-nc-sa/3.0/es/) de Creative Commons



Llenguatge SQL: DCL i extensió procedimental

SQL hostatjat dins un llenguatge de tercera generació..

Contingut

Contingut ii

Capítol 1. SQL hostatjat.....	3
1.1. Introducció.....	3
1.2. Llenguatge MySQL	4
1.2.1. Tipus de blocs de codi.....	4
1.3. Variables	6
1.3.1. Declaració (DECLARE)	6
1.3.2. Assignació	6
1.4. Estructures de control de flux	8
1.4.1. Estructures condicionals	8
1.4.2. Instruccions iteratives.....	10
1.5. Subprogrames (procediments i funcions)	12
1.5.1. Introducció	12
1.5.2. Procediments	12
1.5.3. Funcions.....	15
1.6. Events / JOBS.....	18
1.7. Cursors.....	20
1.8. <i>Handlers</i> (Capturadors)	24
1.9. Generació d'excepcions.....	29
1.9.1. SIGNAL	29
1.9.2. RESIGNAL	32
1.10. Disparadors (<i>Triggers</i>)	33
1.10.1. Referències NEW i OLD.....	36
1.11. Gestió d'usuaris i permisos	40
1.11.1. Gestió de comptes d'usuari.....	40
I. Ús de comodins	41
II. Creació d'usuaris	43
III. Modificacions a usuaris.....	46
IV. Esborrat d'usuaris	47
V. Visualitzar el comptes creats	47
VI. BLOGUEIG DE COMPTES	47
1.11.2. Rols (a partir de la versió 8.0)	48
1.11.3. Gestió de permisos	51
I. Assignar permisos - Sintaxi GRANT	53
II. Assignar permisos - Sintaxi GRANT role	54
III. Treure permisos - Sintaxi REVOKE	54
IV. Treure permisos - Sintaxi REVOKE	55
V. Determinar permisos.....	56
1.11.4. Comprovació dels permisos de rol.....	57
1.11.5. Intercambiabilitat d'usuaris i rols	57

Capítol 1. SQL hostatjat

1.1. Introducció

Fins ara hem estudiat i treballat amb la base de dades de forma interactiva, és a dir, l'usuari introdueix una sentència SQL i el SGBD dóna la resposta de forma immediata.

Aquesta forma de treballar no és la millor, tot i que l'SQL és un llenguatge bastant senzill de treballar, no tots els usuaris coneixen ni tenen perquè conèixer aquest llenguatge.

Per tant, el millor és programar les sentències SQL, utilitzant un llenguatge de programació.

Així que, les sentències SQL que treballen de forma hostatjada (també es diu que es treballa de forma integrada, immersa, etc.) utilitzen aquestes sentències en un programa escrit en un llenguatge de programació.

Nosaltres estudiarem l'SQL immers amb MySQL i més endavant SQL Server (T-SQL

Els programes creats amb el llenguatge de programació es poden emmagatzemar a la mateixa base de dades com qualsevol altre objecte.

D'aquesta manera es facilita a tots els usuaris autoritzats de la base de dades que hi puguin accedir i executar el processos programats.

Hi ha un seguit d'avantatges quan es treballa amb PL/SQL:

- Facilitar la distribució del treball.
- La instal·lació i manteniment del programa.
- Reducció del cost de treball.
- Aprofitament del codi / reutilització del codi.

A més a més, cal destacar que els programes s'executen en el servidor i això permet que el SGBD n'optimitzi l'execució.

1.2. Llenguatge MySQL

Referència: <http://dev.mysql.com/doc/refman/5.7/en/stored-programs-views.html>

1.2.1. Tipus de blocs de codi

En MySQL tenim dos tipus de blocs de codi els **procediments** i les **funcions**. Aquests blocs de codi actuen o tenen el mateix comportament que amb un llenguatge de programació.

Tan els procediments com les funcions s'han de crear sobre una base de dades determinada i es guardaran com un objecte més d'aquesta.

PROCEDIMENT

Un procediment és un bloc de codi que té un nom i li podem passar una sèrie d'arguments com a paràmetres. El bloc de codi s'executarà cada vegada que invoquem el nom del procediment.

Sintaxi:

```
DELIMITER //
CREATE PROCEDURE nom_procediment ( arg1 tipus1, arg2 tipus2 ...)
BEGIN
    Declaració de variables
    Declaració de cursors
    Declaració de handlers
    Instruccions acabades amb punt i coma(;)
END
//
DELIMITER ;
```

Per cridar un procediment farem servir la sentència:

```
CALL nom_procediment(arguments) ;
```

FUNCIÓ

Similar a un procediment, amb la diferència de que es passa un paràmetre (com a mínim) i retorna un valor.

Sintaxi:

```
DELIMITER //  
CREATE FUNCTION funcio_test(arg1 tipus1,...) RETURNS tipusRetorn  
BEGIN  
    Declaració de variables  
    Declaració de cursors  
    Declaració de handlers  
    Instruccions acabades amb punt i coma(;  
    RETURN <expressió de retorn>;  
END  
//  
DELIMITER;
```

Fixeu-vos amb l'ús de la comanda DELIMITER. Dins de la funció tenim punts i comes per separar les instruccions, però el punt i coma li diria al servidor que ha d'executar les instruccions, però encara no ha acabat la funció i pot confondre al servidor.

Amb DELIMITER // (es pot posar els caràcters que un vulgui), substitueix momentàneament el punt i coma com a forma d'acabar les sentències. Així es permet finalitzar d'escriure el procediment (o funció) de forma completa.

1.3. Variables

1.3.1. Declaració (DECLARE)

Per tal de declarar una variable local dins d'un procediment o funció utilitzarem la sentència DECLARE.

Sintaxi:

```
DECLARE var1 [, var2] ... tipus [DEFAULT valor]
```

On:

- **tipus** és qualsevol tipus suportat per MySQL (VARCHAR, INT, DECIMAL, TEXT,...). Són els mateixos tipus de dades que s'utilitzen per crear els camps de les taules.
- **valor** per defecte que podem assignar a la variable/s local/s declarada/es. Aquest valor pot ser una expressió i si no li podem el valor inicial serà NULL.

Exemples:

```
DECLARE nom VARCHAR(20);  
DECLARE i INT(4);  
DECLARE i,p,f INT(4) DEFAULT 0;
```

Es recomana que el nom d'una variable no tingui el mateix nom d'una columna d'una taula per tal d'evitar confusió al *PARSER* en el moment de la seva utilització juntament amb una sentència DML.

<http://dev.mysql.com/doc/refman/5.7/en/stored-program-variables.html>

1.3.2. Assignació

Assignació de valors a variables (SET)

Podem canviar el valor d'una variable mitjançant la sentència **SET**

```
SET nom_variable = valor;  
SET nom_variable := valor; ( l'operador "=" és equivlanent a l'operador " :=" )
```

Exemple:

```
DECLARE nom VARCHAR(20);
```

```
SET nom = 'Maria';  
SET nom:= 'Pere';
```

Assignació de valors a variables (SELECTs)

Una variable també pot prendre el valor del resultat d'una consulta que només retorni un registre.

Sintaxi:

```
SET <nom_variable> = (SELECT <variable_sistema>);  
  
SET <nom_variable> = (SELECT de tipus escalar);  
  
SELECT <camps,expressions> INTO <llista_noms_variables>  
FROM ...
```

Exemples:

```
#Assignació de l'usuari actual a la variable nom_usuari  
SET nom_usuari = (SELECT current_user());  
  
#Assignació del salari màxim de la taula empleats a la variable  
MaxSalari  
SET s:=(SELECT MAX(salari) FROM empleats);  
  
#  
SELECT nom,cognoms INTO @vNom, @vCognoms  
FROM clients  
LIMIT 1;
```

És important saber que quan agafem valors a partir d'una consulta per carregar-los en una o més variables, si la consulta retorna més d'una fila ens donarà error en temps d'execució.

Més endavant veurem els **CURSORS** que ens permetran tractar aquest cas i fer iteracions sobre el resultat d'una consulta de vàries files.

1.4. Estructures de control de flux

1.4.1. Estructures condicionals

IF simple

```
IF condició THEN
    instruccions
END IF;
```

condició és qualsevol expressió que retorni cert o fals.

Exemple:

```
IF departament = 134 THEN
    SET salari := salari * 13;
    SET departament := 123;
END IF;
```

IF-ELSE

```
IF condició THEN
    instruccions1
ELSE
    instruccions2
END IF;
```

Exemple:

Tenim un tall de codi que ens comprova si

```
IF saldo>90 THEN
    SELECT "saldo més gran del que s'esperava.";
ELSE
    IF saldo>0 THEN
        SELECT "Saldo menor del que s'esperava";
    ELSE
        SELECT "Saldo negatiu";
    END IF;
END IF;
```

Una alternativa a aquesta instrucció és:

```
IF condició1 THEN
    instruccions1
ELSIF condició2 THEN
    instruccions2
[ELSIF.... ]
ELSE
    instruccionsElse
END IF;
```

Exemple:

```
IF saldo>90 THEN
    SELECT "Saldo més gran del que s'esperava.";
ELSIF saldo>0 THEN
    SELECT "Saldo menor del que s'esperava";
ELSE
    SELECT "Saldo NEGATIU";
END IF;
```

Intrucció CASE

```
CASE variable
    WHEN expressió1 THEN instruccions1
    WHEN expressió2 THEN instruccions2
    ...
    [ELSE resultatElse]
END;
```

Exemple:

```
DECLARE text VARCHAR(50);

CASE actitud
    WHEN 'A' THEN SET text := 'Molt bona';
    WHEN 'B' THEN SET text := 'Bona';
    WHEN 'C' THEN SET text := 'Normal';
    WHEN 'D' THEN SET text := 'Dolenta';
    ELSE SET text := 'Desconeguda';
END CASE;
```

```
CASE
  WHEN expressió1 THEN instruccions1
  WHEN expressió2 THEN instruccions2
  ...
  [ELSE resultatElse]
END CASE;
```

Exemple:

```
DECLARE text VARCHAR(50);

CASE
  WHEN actitut = 'A' THEN SET text := 'Molt bona';
  WHEN actitut = 'B' THEN SET text := 'Bona';
  WHEN actitut = 'C' THEN SET text := 'Normal';
  WHEN actitut = 'D' THEN SET text := 'Dolenta';
  ELSE SET text := 'Desconeguda';
END CASE;
```

1.4.2. Instruccions iteratives

WHILE

L'estructura WHILE es comporta com en moltes estructures iteratives d'altres llenguatges. Les instruccions de dins el bloc WHILE s'aniran executant fins que la condició sigui falsa.

Sintaxi:

```
WHILE condició DO
  instruccions
END WHILE;
```

Exemple:

```
DECLARE c INT DEFAULT 1;

WHILE c <= 10 DO
  SELECT c;
  SET c := c + 1;
END WHILE;
```

REPEAT

L'estructura REPEAT es comporta com en moltes estructures iteratives d'altres llenguatges. Les instruccions de dins el bloc REPEAT s'aniran executant fins que la condició sigui certa.

Sintaxi:

```
REPEAT
  instruccions
UNTIL condicio
END REPEAT;
```

Exemple:

```
BEGIN
SET x = 0;
REPEAT
  SELECT x;
  SET x = x + 1;
UNTIL x > 10 END REPEAT;
```

1.5. Subprogrames (procediments i funcions)

1.5.1. Introducció

Un subprograma, procediment emmagatzemat o *stored procedure* és un fragment de programa que es troba emmagatzemat al servidor de la base de dades i permet la seva execució per qualsevol usuari amb permisos i des de qualsevol ubicació amb connexió al SGBD.

Els stored procedures poden ser:

Procediments: accions que no retornen cap valor.

Funcions: funcions que retornen un valor.

Per cada procediment i funció crearem un fitxer .sql a on el nom del fitxer agafarà el nom del subprograma.

Dels subprogrames se'n comprova la sintàxi i s'emmagatzemen a la base de dades. No es comprova si les taules existeixen o si l'operació es podrà realitzar o no.

<http://dev.mysql.com/doc/refman/5.7/en/create-procedure.html>

<http://www.bit-util.com/mysql/stored-procedures.html#show-create-procedure>

<http://es.paperblog.com/stored-procedure-en-mysql-parte-1-561762/>

1.5.2. Procediments

Un procediment és un bloc de codi al que se li assigna un nom. Un procediment es crea perquè faci una determinada tasca.

Gràcies a ells es pot aconseguir fer una reutilització eficient del codi, ja que es pot invocar al procediment les vegades que faci falta des d'un altre subprograma

Un cop estan guardats també es poden modificar.

La seva sintaxi és:

```
CREATE PROCEDURE sp_Nom ([def_param[,def_param, ...]])
BEGIN
    seccio_declaracions
    instruccions
END;
```

On, *def_param*:

```
[ IN | OUT | INOUT ] param_nom tipus
```

on:

IN: Indica que el paràmetre es passarà per valor. Es passa el valor dins del procediment i aquest es pot modificar dins l'àmbit de la funció sense que aquests canvis siguin visibles per qui ha cridat el procediment.

OUT: Si indiquem el paràmetre OUT el valor serà NULL a l'inici del procediment i qui ha cridat aquest procediment podrà obtenir-ne el valor un cop s'hagi acabat la crida del procediment.

INOUT: S'inicialitza per qui crida el paràmetre i es pot modificar dins el procediment. Vindria a ser un paràmetre per referència.

Exemple:

```
DELIMITER //
```

```
CREATE PROCEDURE simpleproc (OUT param1 INT)
BEGIN
    SELECT COUNT(*) INTO param1 FROM t;
END
//
DELIMITER ;

...
DECLARE var INT DEFAULT 0;

CALL simpleproc(var);
...
```

Exemple:

```
USE rrhh;

DROP PROCEDURE IF EXISTS sp_AfegirDepartament;

DELIMITER //
CREATE PROCEDURE sp_AfegirDepartament(pDepId INT, pNomDep
VARCHAR(40), pLocId INT)
BEGIN
    INSERT INTO departaments(departament_id,nom,localitzacio_id)
    VALUES (pDepId, pNomDep, pLocId );
END
//

CALL sp_AfegirDepartament(100, 'Nom Departament', 1500);
```

Exemple per veure la diferència dels modificadors IN, OUT I INOUT:

```
DELIMITER //
DROP PROCEDURE IF EXISTS proc2//
CREATE PROCEDURE proc2 (IN p1 INT, OUT p2 INT, INOUT p3 INT)
BEGIN
    SET p1 = p1 + 10;
    SET p2 = p2 + 10;
    SET p3 = p3 + 10;
END
//
DROP PROCEDURE IF EXISTS proc1//
CREATE PROCEDURE proc1()
BEGIN
    DECLARE v1,v2,v3 INT DEFAULT 0;
    SET v1 = 0;
    SET v2 = 1;
    SET v3 = 3;

    CALL proc2(v1,v2,v3);

    SELECT v1,v2,v3;

END
//

#quin valors es mostraran per pantalla?
CALL proc1();
```

Veure la definició de com s'ha creat el procediment

```
SHOW CREATE PROCEDURE <nom_proc>
```

Obtenir informació respecte el procediment: data de creació, usuari, base de dades a on està,....

```
SHOW PROCEDURE STATUS <nom_proc>
```

Eliminar un procediment

```
DROP PROCEDURE <nom_proc>
```

Nota: Si volem modificar un procediment cal borrar-lo i tornar-lo a crear

1.5.3. Funcions

Les funcions són un tipus especial de subprogrames que permeten retornar un valor. Tot el que s'ha comentat en l'apartat anterior és vàlid amb les funcions excepte el tipus de paramètres que només poden ser de tipus IN i que cal retornar un valor al final de la funció.

A partir de la versió 8.0 és important indicar que les funcions no poden modificar la BD i cal indicar-li mitjan

La seva sintaxi és:

```
CREATE FUNCTION sp_Nom ([def_param[,def_param, ...]]) RETURNS tipus
    característiques
BEGIN
    seccio_declaracions
    instruccions
END;
```

Només pot ser de tipus IN, per tant no cal especificar el tipus de paràmetre (IN/OUT/INOUT).

On, *def_param* és:

```
param_nom tipus
```

tipus pot ser qualsevol tipus de dades per la creació de camps dins d'una BD MySQL.

On, *característiques* pot ser:

```
COMMENT 'text'
| [NOT] DETERMINISTIC
| { CONTAINS SQL | NO SQL | READS SQL DATA | MODIFIES SQL DATA }
```

`COMMENT 'text'` Com a característiques de la funció hi podem posar un comentari (aquest comentari es guardarà juntament amb la funció dins de la BD i que podrem consultar).

`[NOT] DETERMINISTIC` : Indiquem si la funció és determinista o no. Determinista significa que la funció sempre retornarà el mateix per els mateixos paràmetres entrats. Això en el SGBD li és útil per una banda optimitzar recursos i realitzar les operacions dins de sistemes de rèplica, etc... **Nota:** MySQL no comprova si el contingut és DETERMINISTA O NO, per tant podem crear funcions no deterministes, però indicar-li en el MySQL que si que ho són. Això pot provocar problemes de rendiment i errors inesperats.

`CONTAINS SQL | NO SQL | READS SQL DATA | MODIFIES SQL DATA`: Si indiquem que la funció no és determinista li hem d'indicar perquè no ho és mitjançant algun d'aquests paràmetres:

- CONTAINS SQL (valor per defecte): Hi ha codi, però no lectura/escriptura a la BD.
- NO SQL: No hi ha codi SQL
- READS SQL DATA: El nostre codi SQL llegeix dades de la BD.
- MODIFIES SQL DATA: El nostre codi modificarà dades de la nostra BD

Important: Un gran avantatge de les funcions és la possibilitat de utilitzar-les dins d'instruccions SQL.

Nota: En MySQL **no** es poden utilitzar sentències DDL dins d'una funció

```
USE rrhh;

#Opcionalment podem eliminar la funció si ja existeix.
DROP FUNCTION IF EXISTS sp_MaxSalari;

DELIMITER //
CREATE FUNCTION sp_MaxSalari(pDepId INT) RETURNS FLOAT
    NOT DETERMINISTIC READS SQL DATA

BEGIN

DECLARE max_salari FLOAT;

    SELECT MAX(salari) INTO max_salari
    FROM empleats
    WHERE dep_id = pDepId;

    RETURN max_salari;
END
//

#Cridem a la funció.
SELECT sp_MaxSalari();
```

També tenim una sèrie de crides per veure informació d'una funció, esborrar-la etc..

Veure la definició de com s'ha creat el procediment

```
SHOW CREATE FUNCTION <nom_funcio>
```

Obtenir informació sobre el procediment: data de creació, usuari, base de dades a on està,....

```
SHOW FUNCTION STATUS <nom_funcio>
```

Eliminar un procediment

```
DROP FUNCTION <nom_funcio>
```

Nota: Si volem modificar una funció cal borrar-la i tornar-la a crear.

També es poden afegir funcions dins del codi del MySQL.

<http://dev.mysql.com/doc/refman/8.0/en/adding-functions.html>

1.6. Events / JOBS

Moltes vegades ens interessa executar una instrucció, un tall de codi o un procediment cada cert temps.

Per exemple volem que cada dia a les 2:00 es realitzi una operació sobre certs registres de la BD, per qüestions de manteniment.

Per exemple:

Imaginem que tenim un BD normalitzada de resultats esportius. Tenim una taula a on hi guardem les classificacions de les diferents competicions. Aquest càlcul seria molt pesat si el volguéssim executar cada vegada que volguéssim consultar la classificació d'una competició. Per tant mantenim una taula a on hi ha les classificacions ja calculades. Aquest càlcul potser el podem realitzar 1 cop cada cert temps.

Per dur a terme aquesta necessitat molts SGBD incorporen un mecanisme per fer-ho.

En MySQL tenim l'objecte EVENT per complir amb aquesta necessitat.

La seva sintaxi és:

```
CREATE EVENT [IF NOT EXISTS] event_name
  ON SCHEDULE schedule
  [ON COMPLETION [NOT] PRESERVE]
  [ENABLE | DISABLE | DISABLE ON SLAVE]
  [COMMENT 'comment']
DO event_body;
```

Nota: Quan l'esdeveniment expira, o sigui, quan la data/hora en la qual se li ha dit que s'havia d'executar ja ha passat aquest s'esborra automàticament.

On:

- **event_name:** Nom que li volem donar al Job/Tasca programada/Event
- **schedule:** Definició de quan i amb quina freqüència s'ha d'executar.
- **ON COMPELTION PRESERVE:** Ens serveix per manteir l'esdeveniment creat després que que aquest hagi expirat.
- **[ENABLE | DISABLE | DISABLE ON SLAVE]:** Mitjançant aquestes opcions podem activar o desactivar l'EVENT sense haver-ho d'esborrar. Si l'esdeveniment està desactivat no s'executarà. També podem només desactivar-lo pels esclaus.
- **COMMENT:** Podem afegir un comentari a on explica una mica el motiu i rol d'aquest Job/Tasca.
- **event_body:** Què ha d'executar (Normalment utilitzarem crides a procediments emmagatzemats.

On, *schedule* té la següent definició:

```
schedule:
  AT timestamp [+ INTERVAL interval] ...
  | EVERY interval
  [STARTS timestamp [+ INTERVAL interval] ...]
  [ENDS timestamp [+ INTERVAL interval] ...]

interval:
  quantity {YEAR | QUARTER | MONTH | DAY | HOUR | MINUTE |
            WEEK | SECOND | YEAR_MONTH | DAY_HOUR | DAY_MINUTE
            | DAY_SECOND | HOUR_MINUTE | HOUR_SECOND | MINUTE_SECOND}
```

Exemples:

```
/* Event que s'executar d'aquí una hora */
CREATE EVENT myevent
  ON SCHEDULE AT CURRENT_TIMESTAMP + INTERVAL 1 HOUR
  DO
    UPDATE myschema.mytable SET mycol = mycol + 1;

/* Event que s'executarà en una data i hora concrets */
CREATE EVENT e_totals
  ON SCHEDULE AT '2006-02-10 23:59:00'
  DO
    INSERT INTO test.totals VALUES (NOW());

/* Volem esborrar les session dels usuaris cada hora*/
CREATE EVENT e_hourly
  ON SCHEDULE EVERY 1 HOUR
  COMMENT 'Clears out sessions table each hour.'
  DO
    DELETE FROM site_activity.sessions;
```

Nota: Si creem un *Event* amb data anterior, donarà un Warning i s'esborrà.

1.7. Cursors

<http://dev.mysql.com/doc/refman/8.0/en/cursors.html>

Fins ara sabem com capturar valors i posar-los en variables, però tenim la limitació que aquest sistema només ens funciona per consultes que retornen una sola fila.

El més normal però, serà que necessitem fer un recorregut sobre un resultat d'una consulta per fer el tractament que calgui per cada registre obtingut.

Els cursors permeten recórrer els diferents registres d'una consulta un a un, i com a característiques en el MySQL tenen que són de **només lectura**, i **no scrollable** (seqüencial).

Aquest tipus d'estructura només s'ha de fer servir quan ens sigui impossible realitzar el que volem directament amb una sentència DML. **Els cursors penalitzen el rendiment del sistema**. Per exemple si volem augmentar el salari dels empleats d'un cert departament no ho farem mitjançant cursors sinó a través d'una sentència UPDATE.

Per poder utilitzar un cursor, caldrà seguir els quatre passos següents:

1. Declarar el cursor

```
DECLARE cursor_name CURSOR FOR select_statement
```

La declaració del cursor s'ha de realitzar en l'apartat de declaracions dins del subprograma (procediment i funció).

2. Obrir el cursor

```
OPEN cursor_name
```

Per utilitzar el cursor utilitzarem la instrucció OPEN per obrir-lo. D'alguna manera la instrucció OPEN prepara/executa la sentència SQL declarada en el punt 1.

3. Iterar sobre cada registre del cursor (realitzar-ne el recorregut)

```
FETCH cursor_name INTO var_name [, var_name] ...
```

Per recuperar els valors del registre actual cal utilitzar la instrucció FETCH. Aquesta instrucció recupera el següent registre dels resultats de la instrucció SELECT i deixa els valors a les variables indicades.

Si no hi ha més registres disponibles llança una excepció amb SQLSTATE '02000' o NOT FOUND. Per detectar aquest final podem definir un controlador (Handler).

4. Tancar el cursor

```
CLOSE cursor_name
```

La instrucció CLOSE ens serveix per tancar el cursor i alliberar els recursos utilitzats per aquest.

Si el cursor no el tanquem explícitament, el cursor es tancarà al final del bloc BEGIN ... END el qual s'hagi declarat.

Salterà un error si intentem tancar un cursor si prèviament aquest no s'ha obert.

Per tal d'alliberar memòria és bona pràctica tancar el cursor un cop no en necessitem més. També es recomanable tancar-lo abans de sortir del subprograma.

Exemple:

Volem fer un "sorteig" dels empleats que es quedaran a fer hores extres. Els "afortunats" els guardarem en una taula `voluntaris`. Per fer-ho, farem un recorregut per la taula `empleats`, i 1 de cada 3 empleats seran els afortunats. Per exemple el 1r, 4t, 7è... per escollir quins són els "afortunats" utilitzarem el paràmetre `pParam`

```
DELIMITER //
DROP PROCEDURE IF EXISTS sp_CrearLlistaVoluntaris//
CREATE PROCEDURE sp_CrearLlistaVoluntaris(IN pParam INT)
BEGIN
    DECLARE num                INTEGER;
    DECLARE fi_cursor          BOOLEAN DEFAULT false;
    DECLARE tmp_id_empleat     INT(4);
    DECLARE tmp_nom            VARCHAR(50);
    DECLARE tmp_cognoms        VARCHAR(80);

    /* Declarem el cursor */
    DECLARE cEmpleats CURSOR FOR SELECT empleat_id,nom,cognoms
                                FROM empleats;

    /* Declarem el handler per gestionar els errors de no trobat
       o fi de select, codis 02xxx */
    DECLARE CONTINUE HANDLER FOR NOT FOUND
    BEGIN
        SET fi_cursor = true;
    END;

    /* Eliminem els registres del sorteig anterior */
    DELETE FROM empleats_voluntaris;

    /* Recorrem la consulta amb el cursor.
       Construïm el bucle amb WHILE
       Quan es produeix l'error que no hi ha més registres a la
       consulta,el handler (gestor d'errors) per a l'error 02000
       (no trobat) posa fi a true.
       Quan fi és true, sortirem del bucle while
    */

    SET num = 0;

    /* Obrim el cursor i fem un primer FETCH */
    OPEN cEmpleats;

    FETCH cEmpleats INTO tmp_id_empleat, tmp_nom, tmp_cognoms;

    WHILE ( fi_cursor = false ) DO

        SET num = (num + 1) ;

        IF num > pParam THEN
            SET num = 1;
        END IF;

        IF (num = 1) THEN
            INSERT INTO empleats_voluntaris(empleat_id,nom,cognoms)
            VALUES (tmp_id_empleat, tmp_nom, tmp_cognoms);
        END IF;
    END WHILE;
END;
```

```
/* Agafem les dades del següent registre */
FETCH cEmpleats INTO tmp_id_empleat, tmp_nom, tmp_cognoms;

END WHILE;

CLOSE cEmpleats;  -- Important tancar el cursor!
END

/* Suposem que la taula de voluntaris ha estat creada prèviament. */
/* Creem la taula */
DROP TABLE IF EXISTS empleats_voluntaris;
CREATE TABLE empleats_voluntaris (
    empleat_id INT(4) PRIMARY KEY,
    nom          VARCHAR(50),
    cognoms      VARCHAR(80)
);
```

1.8. Handlers (Capturadors)

Els *Handlers* els utilitzem per capturar certs esdeveniments que passen en el nostre bloc de codi dins d'un subprograma.

Normalment els utilitzem per capturar errors, excepcions, etc...

Ens permetran fer un tractament especial quan es produeixi un error dins dels nostres subprogrames.

És un mecanisme semblant al que utilitzen certs llenguatges com C#, Java,... pel tractament d'excepcions (try / catch).

En el cas que es produeixi un error/excepció, els handlers serem capaços d'interceptar-lo i actuar en conseqüència.

Quins errors es poden produir? Doncs per exemple inserir un registre duplicat, consultar un registre amb una SELECT i que aquesta no retorni cap dada, etc.

<https://dev.mysql.com/doc/refman/8.0/en/declare-handler.html>

<https://dev.mysql.com/doc/refman/8.0/en/error-handling.html>

<https://dev.mysql.com/doc/refman/8.0/en/error-messages-server.html>

Sintaxi:

```
DECLARE handler_type HANDLER FOR condition_value[,...] sp_statement
```

handler_type:

CONTINUE
| EXIT

condition_value:

SQLSTATE [VALUE] sqlstate_value
| condition_name
| SQLWARNING
| NOT FOUND
| SQLEXCEPTION
| mysql_error_code

on **handler_type** ens indica què farem una vegada tractat l'error:

- **CONTINUE**: Continuarem l'execució del subprgrama a partir del punt de codi següent del que ha produït l'error.
- **EXIT**: Acabarem l'execució del subprograma una vegada tractat l'error

on **condition_value** ens indica quin tipus d'error volem capturar:

- **SQLSTATE [VALUE] sqlstate_value**: Codi de 5 caràcters que indica quin estat SQL s'ha produït. Quan es produeix un error MySQL assigna a la variable

SQLSTATE un codi d'error. Per saber tots els codis d'errors podeu consultar el següents enllaços:

- [Server Error Message Reference](#)
- <http://dev.mysql.com/doc/refman/8.0/en/error-handling.html>
- <http://dev.mysql.com/doc/refman/8.0/en/error-types.html>
- <https://dev.mysql.com/doc/refman/8.0/en/error-messages-server.html>

Els dos primers caràcters d'un valor SQLSTATE indica el tipus d'error:

Class = '00' indica OK.

Class = '01' indica WARNING.

Class = '02' indica “*NOT FOUND*” Aquest tipus d'error apareix en l'ús de CURSORS o també si utilitzem una sentència del tipus `SELECT ... INTO var_list` que no retorni cap fila

Class > '02' indica una EXCEPCIÓ

Per exemple, un SQLSTATE = '23000' significa que s'ha produït un error al intentar inserir un registre duplicat en una taula.

- `condition_name`: És una condició que nosaltres prèviament hem declarat mitjançant [DECLARE...CONDITION](#). Aquest nom de condició pot associar-se amb un codi d'error o amb un SQLSTATE
- `SQLWARNING`: Engloba tots els SQLSTATE que comencen per 01. És a dir tots els que es consideren avisos.
Per exemple el codi 01000 significa que al fer una inserció s'han truncat dades.
- `NOT FOUND`: Engloba tots els SQLSTATE que comencen per 02. Fa referència a quan no es troben dades: al fer una `SELECT` o quan realitzem un `FETCH` d'un cursor i aquest ja estava al final.
- `SQLEXCEPTION`: Engloba tots els codis no tractats per `SQLWARNING` i `NOT FOUND`
- `mysql_error_code`: codi d'error del MySQL mirar [Error Code Ranges](#)

on `sp_statement` indiquem el bloc de codi a executar.

- Si només es tracta d'una sola sentència la podem posar tal qual. Si volem que s'executi un bloc de codi cal tancar-lo dins un `BEGIN ... END`.

Funcions per obtenir els codies o missatges d'error

`mysql_sqlstate()` returns the SQLSTATE value.

`mysql_errno()` returns the MYSQL_ERRNO value.

`mysql_error()` returns the MESSAGE_TEXT value.

Exemples:

```
DROP PROCEDURE IF EXISTS sp_InsertDept;

DELIMITER //
CREATE PROCEDURE sp_InsertDept (pDeptId INT(4), pDeptNom VARCHAR(30),
pLocId INT(4), OUT error BOOL)
BEGIN
    DECLARE usuari VARCHAR(100);
    DECLARE data DATETIME;

    DECLARE CONTINUE HANDLER FOR SQLSTATE '23000' SET error = 1;

    SELECT CURRENT_USER(), NOW() INTO usuari, data;

    INSERT INTO departaments (departament_id,nom,localitzacio_id
        VALUES (pDeptId, pDeptNom, pLocId);

    -- Com que hem definit el HANDLER amb CONTINUE, encara
    -- que casqui la instrucció anterior, un cop tractat
    -- l'error, que en el nostre cas es posar la variable error
    -- a 1, continuarà amb l'execució.

    INSERT INTO log_usuari (usuari, data, taula, accio, valor_pk,
error)
VALUES ( usuari, data, 'DEPARTAMENT', 'INSERT', pDeptId,
error);

END
//
```

```
DROP PROCEDURE IF EXISTS sp_InsertDept;

DELIMITER //
CREATE PROCEDURE sp_InsertDept (pDeptId INT(4), pDeptNom VARCHAR(30),
pLocId INT(4), OUT error BOOL)
BEGIN
    ...
    DECLARE CONTINUE HANDLER FOR SQLException SET error = 1;
    ...
END
//
```

```
DROP PROCEDURE IF EXISTS sp_InsertDept;

DELIMITER //
CREATE PROCEDURE sp_InsertDept (pDeptId INT(4), pDeptNom VARCHAR(30),
pLocId INT(4), OUT error BOOL)
BEGIN
    ...
    DECLARE clau_duplicada CONDITION FOR SQLSTATE '23000';
    DECLARE CONTINUE HANDLER FOR clau_duplicada SET error = 1;
    ...
END
```

```
//  
DROP PROCEDURE IF EXISTS sp_InsertDept;
```

Exemple d'utilitzar un bloc de codi en la resposta del handler

```
DELIMITER //  
CREATE PROCEDURE sp_InsertDept (pDeptId INT(4), pDeptNom VARCHAR(30),  
pLocId INT(4), OUT error BOOL)  
BEGIN  
    ...  
    DECLARE CONTINUE HANDLER FOR SQLEXCEPTION  
    BEGIN  
        SET error = 1;  
        INSERT INTO errors (datahora, usuari, nom)  
            VALUES(NOW(), CURRENTUSER(), 'sp_InsertDept')  
    END;  
    ...  
END  
// //
```

1.9. Generació d'excepcions

Moltes vegades ens interessa poder generar una excepció per tal de qui ha cridat el procediment pugui capturar-la i controlar els errors.

Molts llenguatges de programació orientats a objectes utilitzen la paraula clau `Throw` o `raise` per generar una excepció i que aquesta es pugui capturar mitjançant un `try/catch`.

Dins del MySQL també tenim aquesta capacitat, tot i que és una mica limitada. El que podem fer és, enviar una senyal de tal manera que canviï el valor del `SQLSTATE` i així poder capturar-lo mitjançant un *Handler*.

<https://dev.mysql.com/doc/refman/8.0/en/signal.html>
<http://www.mysqltutorial.org/mysql-signal-resignal/>

1.9.1. SIGNAL

```
SIGNAL condition_value  
[SET signal_information_item  
[, signal_information_item] ...]
```

condicion value

On *condition_value*: Serà el valor d'error que volem retornar. Pot ser:

- Un valor `SQLSTATE` (5 caràcters) o
- El nom d'una condició que prèviament hem declarat mitjançant `DECLARE`
- Hi ha un `SQLSTATE` genèric que és el '45000' (excepció no definida)

```
SQLSTATE [VALUE] sqlstate_value  
| condition_name
```

signal information item

Si volem donar més informació de l'error podem modificar algun d'aquests ítems.

`condition_information_item_name = simple_value_specification`

condition_information_item_name:

```
CLASS_ORIGIN  
| SUBCLASS_ORIGIN  
| MESSAGE_TEXT ← més utilitzat  
| MYSQL_ERRNO ← més utilitzat  
| CONSTRAINT_CATALOG  
| CONSTRAINT_SCHEMA  
| CONSTRAINT_NAME  
| CATALOG_NAME  
| SCHEMA_NAME  
| TABLE_NAME  
| COLUMN_NAME  
| CURSOR_NAME
```

simple_value_specification: Pot ser qualsevol valor literal (un string un enter,...), el valor d'una variable, el retorn d'una funció, etc...

Exemple: Tenim una funció anomenada spDivisio que volem que doni un error si el paràmetre divisor és zero.

```
CREATE FUNCTION spDivisio(pDivident INT,pDivisor INT) RETURNS DECIMAL
DETERMINISTIC
BEGIN
    IF pDivisor = 0 THEN
        SIGNAL SQLSTATE '22012';
    ELSE
        RETURN pDivident/pDivisor;
    END IF;
END;

-- EXAMPLE 2
CREATE FUNCTION spDivisio(pDivident INT,pDivisor INT) RETURNS DECIMAL
DETERMINISTIC
BEGIN
    DECLARE divide_by_zero CONDITION FOR SQLSTATE '22012';
    IF pDivisor = 0 THEN
        SIGNAL divide_by_zero;
    ELSE
        RETURN pDivident/pDivisor;
    END IF;
END;
```

Exemple de **personalització** del missatge d'error i el codi d'error:

Exemple: Agafant l'exemple anterior personalitzarem el missatge d'error.

```
-- EXAMPLE 2
CREATE FUNCTION spDivisio(pDivident INT,pDivisor INT) RETURNS DECIMAL
DETERMINISTIC
BEGIN
    DECLARE divide_by_zero CONDITION FOR SQLSTATE '22012';
    IF pDivisor = 0 THEN
        SIGNAL divide_by_zero
        SET MESSAGE_TEXT = "El divisor no pot ser 0";
    ELSE
        RETURN pDivident/pDivisor;
    END IF;
END;
```

Exemple: Tenim un procediment que permet afegir una línia de factura en un esquema de factures-línies_factura i volem controlar-ne els errors.

```
CREATE PROCEDURE spAfegirLiniaFactura( pFacturaId INT,
                                       pProducteCodi CHAR(4),
                                       pQt TINYINT,
                                       pImport FLOAT)
BEGIN
  DECLARE vHiHaErr TYNINT DEFAULT 0;
  DECLARE vErrMis VARCHAR(240) DEFAULT '';

  --
  -- Comprovem que la factura ens han passat existeixi.
  --
  SELECT COUNT(factura_id) INTO lExisteix
    FROM factures
   WHERE factura_id = pFacturaId;

  IF(vHiHaErr != 1) THEN
    SET vErrMis = CONCAT(vErrMis
                        , 'La factura amb identificador '
                        , pFacturaId,
                        , ' no existeix');
  END IF;

  --
  -- Comprovem que el codi de producte que ens han passat existeixi.
  --
  SELECT COUNT(producte_codi) INTO vHiHaErr
    FROM productes
   WHERE producte_codi = pProducteCodi;

  IF(vHiHaErr != 1) THEN
    SET vErrMis = CONCAT(vErrMis
                        , '\n El producte amb codi '
                        , pProducteCodi
                        , ' no existeix');
  END IF;

  -- Si hi ha hagut algun error executem el signal
  IF(vHiHaErr != 1) THEN

    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = vErrMis,
        MYSQL_ERRNO = 1000;

  ELSE

    INSERT INTO linies_factura(factura_id,prod_codi,qt,import)
      VALUES(pFacturaId, pProducteCodi, pQt, pImport);

  END IF;
END
```

1.9.2. RESIGNAL

MySQL proporciona un altre sentència que funciona de la mateixa manera que el SIGNAL però té petits canvis de comportament:

- **S'ha d'utilitzar en un HANDLER**
- Es poden ometre tots els atributs del RESIGNAL, inclòs el valor SQLSTATE

Mirar els enllaços:

- <http://www.mysqltutorial.org/mysql-signal-resignal/>
- <https://dev.mysql.com/doc/refman/8.0/en/resignal.html>

RESIGNAL Alone

<https://dev.mysql.com/doc/refman/8.0/en/resignal.html> (apartat 14.6.7.4.1 *RESIGNAL Alone*)

RESIGNAL with New Signal Information

RESIGNAL with a Condition Value and Optional New Signal Information

Exemples:

```
// Ús del RESIGNAL s  l
RESIGNAL;

//RESIGNAL amb un signal information canviant els information_items:
RESIGNAL SET signal_information_item [, signal_information_item] ...;

//RESIGNAL amb una condici   I amb una possible nova informaci  :
RESIGNAL condition_value
    [SET signal_information_item [, signal_information_item] ...];
```

Exemple:

```
DELIMITER //
DROP PROCEDURE IF EXISTS spDivisio//
CREATE PROCEDURE spDivisio(IN numerador INT, IN denominador INT, OUT
resultat double)
BEGIN
    DECLARE divisio_per_zero CONDITION FOR SQLSTATE '22012';

    DECLARE CONTINUE HANDLER FOR divisio_per_zero RESIGNAL SET
MESSAGE_TEXT = 'Divisio per zero / El Denominador no pot ser zero';
    --
    IF denominador = 0 THEN
        SIGNAL divisio_per_zero;
    ELSE
        SET resultat := numerador / denominador;
    END IF;
END
```

1.10. Disparadors (*Triggers*)

Els disparadors (*triggers*) són objectes dins d'una base de dades que s'associen a una taula i s'activen quan passa un esdeveniment (inserció, modificació, esborrat). Dins d'aquest disparador hi haurà un bloc de codi que podrem programar i s'executarà automàticament quan passi algun dels esdeveniments anteriors.

Els disparadors s'assemblen als procediments i funcions, ja que són blocs de codi amb un nom i amb les seccions: declarativa, executable i de gestió d'excepcions.

Per què són necessaris els *triggers*?

- Mantenir camps calculats a on intervenen més d'una taula.
- Per mantenir restriccions d'integritat complexes, que no és possible definir-les en el moment de crear la taula. Per exemple, DEFAULT NOW()
- Portar un control sobre la informació que conté una taula, registrant els canvis realitzats i/o qui ha fet els canvis i/o quan.
- Executar automàticament altres programes (procediments o funcions) en el moment en que es fan canvis sobre les dades de la taula.

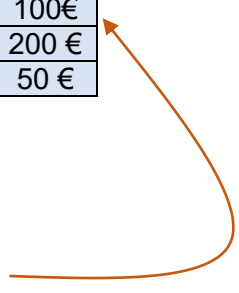
<http://dev.mysql.com/doc/refman/5.7/en/trigger-syntax.html>

<http://dev.mysql.com/doc/refman/5.7/en/create-trigger.html>

Exemple: Imaginem que tenim una taula de factures amb un total i volem que aquest total sigui calculat a través de les línies de factura. Si s'afegeix una nova, es modifica o s'esborra caldrà modificar el valor del camp total de les factures afectades.

FACTURES			
factura_id	client_id	data_factura	total
1	100	05/05/2019	100€
2	100	06/05/2019	200 €
3	101	05/08/2019	50 €

LINIES_FACTURA			
factura_id	num_linia	...	import
1	1	...	25€
1	2	...	50 €
1	3	...	25 €
...	...	...	...



Per exemple si modifiquem l'import de la línia de factura número 3 de la factura_id 1 amb un import de 75€ el total de la factura passarà de 100€ → a 150€.

LINIES_FACTURA			
factura_id	num_linia	...	import
1	1	...	25€
1	2	...	50 €
1	3	...	25 € 75€
...	...	...	...

FACTURES			
factura_id	client_id	data_factura	total
1	100	05/05/2019	100€ 150€
2	100	06/05/2019	200 €
3	101	05/08/2019	50 €

Sintaxi:

```
CREATE TRIGGER <nom_trigger> {BEFORE | AFTER} {INSERT | UPDATE |  
DELETE} ON <nom_taula> FOR EACH ROW [trigger_order] <nom_altre_trigger>  
BEGIN  
  <codi del trigger>  
END;
```

A on:

- {BEFORE | AFTER}: Indica el moment en què s'executarà el *trigger*. Indiquem si es dispararà abans de fer efectiu l'esdeveniment DML o després.
- {INSERT | UPDATE | DELETE}: Indica quin esdeveniment DML produirà el llançament del *trigger*.
- *trigger_order*: {FOLLOWS | PRECEDES } *nom_altre_trigger*: Indiquem l'ordre de precedència d'un trigger envers algun altre.
Nota: Aquesta sentència és vàlida a partir del MySQL 5.7

Exemple: Volem executar un procediment emmagatzemat cada vegada que realitzem un INSERT a la taula de comptes_corrents.

```
DELIMITER //  
CREATE TRIGGER trg_ActualitzaSaldos BEFORE INSERT  
  ON comptes_corrents FOR EACH ROW  
BEGIN  
  CALL sp_ActualitzarSaldos();  
END//
```

També podem establir una nomenclatura alhora d'anomenar els triggers. La nomenclatura proposada podria ser: *trg_<nomTrigger>*

En MySQL la sentència del disparador s'executarà per cada registre afectat per la instrucció DML. D'aquí ve la clàusula **FOR EACH ROW**. Aquesta clàusula en altre SGBD pot tenir diferents valors.

Nota: No es poden crear triggers que facin disparar a ells mateixos. És a dir:

```
DELIMITER //  
CREATE TRIGGER trg_ActualitzaSaldos BEFORE INSERT  
  ON t1 FOR EACH ROW  
BEGIN  
  INSERT INTO t1 (c1) VALUES(NEW.c1);  
END//  
  
INSERT INTO t1 (c1) VALUES(1);
```

Exemple: A la BD de rrhh volem tenir una taula num_empleats a on hi hagi el número d'empleats de la nostra BD i volem que aquest valor es gestioni de forma automàtica.

```
DROP TABLE IF EXISTS num_empleats;
CREATE TABLE num_empleats(
    quantitat INTEGER UNSIGNED
);

INSERT INTO num_empleats(quantitat)
VALUES(SELECT count(*) from empleats);

-- Trigger per controlar quan s'afegeix un empleat
DELIMITER //
DROP TRIGGER IF EXISTS trg_ActNumEmpleats_insert
//
CREATE TRIGGER trg_ActNumEmpleats_insert AFTER INSERT
ON empleats FOR EACH ROW
BEGIN
    UPDATE num_empleats
    SET quantitat = quantitat + 1;
END//

--Triger per controla quan s'esborra un empleat
DELIMITER //
DROP TRIGGER IF EXISTS trg_ActNumEmpleats_delete
//
CREATE TRIGGER trg_ActNumEmpleats_delete AFTER DELETE
ON empleats FOR EACH ROW
BEGIN
    UPDATE num_empleats
    SET quantitat = quantitat - 1;
END//
```

1.10.1. Referències NEW i OLD

Quan s'executen els *triggers*, cal tenir en compte que es modifiquen valors antics (OLD) per canviar-los per valors nous (NEW).

Dins del bloc de codi tenim els **pseudoregistres** anomenats **NEW** i **OLD** que permeten fer referència als valors nous i antics respectivament.

En el cas d'un UPDATE, el registre **NEW** conté els nous valors del registre de la taula que es vol canviar, i el registre **OLD** conté els valors antics.

En el cas d'un INSERT, el registre **OLD** no existeix.

En el cas d'un DELETE, el registre **NEW** no existeix.

Així, **NEW.nom** faria referència al nou nom que s'assigna a una determinada taula i **OLD.nom** al vell.

Els valors dels camps del registre **NEW** es pot modificar com si fos una variable més:

```
SET NEW.camp = valor;
```

El registre OLD és només de lectura.

Exemple: Volem crear un *trigger* per la taula anomenada `opera` de manera que quan s'afegeixi un nou registre comprovi que el valor del camp `idioma` està dins del rang de valors establerts ('I','F' o 'A'). En cas contrari el disparador assignarà el valor 'I' al camp `idioma`.

```
DELIMITER //
CREATE TRIGGER trg_ComprovaIdioma BEFORE INSERT
  ON opera FOR EACH ROW
BEGIN
  IF NEW.idioma != 'I'
    AND NEW.idioma != 'F'
    AND NEW.idioma != 'A' THEN
    SET NEW.idioma='I';
  END IF;
END//
```

Exemple: Tenim una taula anomenada `emails_enviats` a on a part de molts altres camps es guardarà la data i hora conforme aquest e-mail va ser enviat.

```
DELIMITER //
CREATE TRIGGER trg_EmailsEnviatsDefault BEFORE INSERT
    ON emails_enviats FOR EACH ROW
BEGIN
    IF NEW.datahora IS NULL THEN
        NEW.datahora = CURRENTDATE();
    END IF;
END//
```

També podem llançar una excepció en un *trigger* perquè la comanda DML no s'executi. Per exemple si ens passen un valor erroni a la sentència DML llançarem un SIGNAL.

Exemple: Volem tenir un *TRIGGER* que ens comprovi abans de fer un INSERT a la taula d'alumnes si el camp `data_naix` és correcte. Serà correcte quan el valor de la data de naixement que ens passin de l'alumne sigui inferior a la data d'avui. Si no ens passen valor inicialitzarem aquesta amb la data d'avui.

Trigger

```
DELIMITER //

DROP TRIGGER IF EXISTS trg_AlumneInsert//

CREATE TRIGGER trg_AlumneInsert BEFORE INSERT ON alumnes
FOR EACH ROW
BEGIN
    IF (NEW.data_naix > CURDATE()) THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = "Error: La data de
naixement no pot ser superior a avui.";
    ELSEIF (NEW.data_naix IS NULL) THEN
        SET NEW.data_naix = CURDATE();
    END IF;
END
```

Exemple

```
mysql> INSERT INTO alumnes (nom,cognom1,data_naix)
VALUES ('Pere', 'Pi', CURDATE()+1));

ERROR 1644 (45000): Error: La data de naixement no pot ser superior a
avui
```

Exemple amb un procediment

```
DROP PROCEDURE IF EXISTS afegirAlumne//
CREATE PROCEDURE afegirAlumne(pNom VARCHAR(25), pCognom1 VARCHAR(35)
                             ,pDataNaix DATE, OUT pError BOOLEAN)
BEGIN
    DECLARE CONTINUE HANDLER FOR SQLSTATE '45000' SET pError = true;

    SET pError = false;

    INSERT INTO alumnes (nom,cognom1,data_naix)
        VALUES (pNom, pCognom1, pDataNaix);
END
//

DELIMITER ;
```

Usos del procediment

```
SET @pError = false;
CALL afegirAlumne('Pere', 'Pi', CURDATE(), @pError);

SELECT @pError;
+-----+
| Error          |
+-----+
| No hi ha error |
+-----+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)
```

```
SET @pError = false;
CALL afegirAlumne('Pere', 'Pi', CURDATE()+1, @pError);

SELECT @pError;
+-----+
| Error          |
+-----+
| Hi ha error    |
+-----+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)
```

També podríem haver afegit un SIGNAL en el procediment.

```
DROP PROCEDURE IF EXISTS afegirAlumne//
CREATE PROCEDURE afegirAlumne(pNom VARCHAR(25), pCognom1 VARCHAR(35)
                             ,pDataNaix DATE)
BEGIN
    DECLARE hiha_error BOOLEAN DEFAULT false;
    DECLARE CONTINUE HANDLER FOR SQLSTATE '45000' SET hiha_error =
true;

    INSERT INTO alumnes (nom,cognom1,data_naix)
        VALUES (pNom, pCognom1, pDataNaix);

    IF hiha_error THEN
        SIGNAL SQLSTATE '99999' SET MESSAGE_TEXT = "Error: La data de
naixement no pot ser superior a avui.";
    END IF;

END
//

DELIMITER ;

mysql> CALL afegirAlumne('Pere', 'Piaa',adddate(curdate(), interval 1
day));

Sortida

ERROR 1644 (99999): Error: La data de naixement no pot ser superior a
avui.
```

1.11. Gestió d'usuaris i permisos

<https://dev.mysql.com/doc/refman/8.0/en/user-account-management.html>

Tot sistema d'informació ens interessa tenir algun mecanisme per tal de donar accés a certes persones i controlar quines accions sobre quines dades poden realitzar. Per exemple, ens interessa crear un usuari del tipus **"read-only"** el qual només pugui realitzar operacions DML de consulta sobre la nostra BD.

Mitjançant aquest sistema de gestió d'usuaris i permisos podem controlar **qui pot fer què**.

1.11.1. Gestió de comptes d'usuari

<https://dev.mysql.com/doc/refman/8.0/en/account-names.html>

<https://dev.mysql.com/doc/refman/8.0/en/account-management-statements.html>

Sintaxi de gestió d'usuaris i permisos

<https://dev.mysql.com/doc/refman/5.7/en/account-management-sql.html>

La definició de comptes d'usuari permet identificar la persona i la màquina que ha realitzat una connexió a una base de dades.

Important:

El compte d'usuari per MySQL està format per un *nom d'usuari* i un *hostname*

El seu format és: 'nom_usuari'@'host'

- **nom_usuari.** És l'identificador de l'usuari (per exemple: professor) com a màxim pot tenir una longitud de 32 caràcters a partir de la versió 5.7.8 i 16 caràcters per versions anteriors.
- **host.** És la màquina des d'on es pot connectar aquest usuari
 - localhost : l'usuari només es connecta des del mateix servidor
 - '192.168.15.30' : l'usuari només es pot connectar des d'una ip determinada.
 - maquina.domini.cat: el nom de la màquina (Cal que els servidor sàpiga resoldre el nom del domini).
 - '%' : l'usuari es pot connectar des de qualsevol ordinador
 - Subxarxes. '192.168.15.30/255.255.0.0'. Es permet l'accés des de la subxarxa
 - Subxarxes. '192.168.%.%'. Igual que l'anterior, però utilitzant comodins estil LIKE, per noms de màquina o per IPs.

Cas especial: usuari anònim '@%' (usuari buit i host qualsevol)

Exemples d'usuari serien:

```
'professor'@'localhost'  
'professor'@'192.168.15.30'  
'professor'@'%'
```

Important: El nom d'usuari i nom del host han d'estar tancats amb un símbol de caràcter (coma, doble coma, o accent obert) de forma separada.

Per exemple:

```
'professor@localhost' #MySQL ho assumiria com el nom d'usuari
```

MySQL enten l'anterior usuari com:

```
'professor@localhost'@'%'
```

I. Ús de comodins

MySQL **no suporta comodins** dins dels noms d'**usuari**, però sí en el **nom del host**.

Exemples:

```
# Qualsevol host dins del domini exemple.com  
'user_name'@'%.example.com'  
  
#Qualsevol host dins de la xarxa de tipus C 192.168.1  
'user_name'@'192.168.1.%'  
  
#Qualsevol host dins de la xarxa de tipus B 192.168  
'user_name'@'192.168.%.%'  
  
#Qualsevol host dins de la xarxa de tipus A 192  
'user_name'@'192.%.%.%'  
  
#Qualsevol host  
'user_name'@'%'
```

També podem utilitzar adreces IPv4 o IPv6 per referenciar-nos al nom del host.

La IP '127.0.0.1' indicarà adreça de loopback en IPv4 i la ':::1' indicarà l'adreça de loopback en IPv6.

Si utilitzem IPv4 podem també especificar la màscara de xarxa:

```
CREATE USER 'usuari'@'192.58.197.0/255.255.255.0';
```

Quan MySQL es troba que l'intent d'identificació d'un usuari pot correspondre a més d'un compte d'usuari segueix les següents pautes per resoldre-ho:

1. Cada vegada que el servidor llegeix la taula **user** a la memòria, ordena les files.
2. Quan un client s'intenta connectar, el servidor mira a través dels registres en aquest ordre.
3. El servidor utilitza la primera fila que coincideix amb el hostname del client i nom d'usuari.

El mètode d'ordenació és el següent:

- El servidor ordena les files amb els valors de **host** més específics primer.
 - El noms literals per hostnames
 - Les IPs per hostnames. (L'especificitat d'una adreça IP no es veu afectada pel fet que tingui màscara de xarxa o no.
 - El patró '%' significa "qualsevol host" i és el menys específic. La cadena buida "" també significa "qualsevol host", però en aquest cas va després del '%'.
- Les files amb el mateix valor de **host** s'ordenen amb els valors de **user** més específic
- Per les files amb els mateixos valors de **host** i **user** l'ordre és indeterminat.

Exemple: Si suposem que tenim aquests valors entrats a la taula **mysql.user**

Host	User	...
%	root	...
%	jeffrey	...
localhost	root	...
localhost		...

Quan el servidor llegeix la taula a memòria ordenarà les files utilitzant les regles anteriors quedant el resultat:

Host	User	...
localhost	root	...
localhost		...
%	jeffrey	...
%	root	...

Exemple2: Quin seria l'ordre si el contingut de la taula user fós el següent?

```
+-----+-----+
| Host          | User      | ...
+-----+-----+
| %             | jeffrey   | ...
| girona.loc.cat |          | ...
+-----+-----+
```

II. Creació d'usuaris

<https://dev.mysql.com/doc/refman/5.7/en/create-user.html>

Sintaxi

```
CREATE USER [IF NOT EXISTS]
    <nom_usuari> [ opcio_auth],
    <nom_usuari> [opcio_auth],...
DEFAULT ROLE <rol [, rol],...
[REQUIRE {NONE | opcio_ssl/tls }]
[WITH opcio_rekurs [opcio_rekurs] ...]
[opcio_pwd];
```

On:

opcio_auth: opcions d'autenticació

```
opcio_auth: {
    IDENTIFIED BY 'clau en clar'
    | IDENTIFIED BY PASSWORD 'clau_hash' (DEPRECATED)
    | IDENTIFIED WITH auth_plugin
    | IDENTIFIED WITH auth_plugin BY 'auth_string'
    | IDENTIFIED WITH auth_plugin AS 'hash_string'
}
```

Cal mirar els diferents pluguins d'autenticació que podem utilitzar:
<https://dev.mysql.com/doc/refman/5.7/en/authentication-plugins.html>.

La característica de poder autenticar els usuaris mitjançant pluguins permet relegar el procés de validació a servidors externs (LDAP, ...)
<https://dev.mysql.com/doc/refman/5.7/en/pluggable-authentication.html>

El pluguin per defecte és: `mysql_native_password`, excepte que canviem la variable de sistema `'default_authentication_plugin'`

Exemple:

```
CREATE USER 'alumne'@'localhost' IDENTIFIED BY 'patata';
```

opció ROLE (A partir de la versió 8.0): Aquesta clàusula defineix quins rols tindrà actius el compte d'usuari.

Aquests rols no necessàriament han d'existir en el moment de crear l'usuari.

```
CREATE USER 'joe'@'10.0.0.1' DEFAULT ROLE administrator, developer;
```

opcio_ssl/tls: opcions de xifrat en la connexió:

```
| NONE  
| SSL  
| X509  
...
```

opcio_rekurs (A partir de la versió 5.7): Tots els usuaris poden tenir opcions de recurs. Per exemple número de consultes per hora, número màxim de connexions que pot establir l'usuari,...

```
opcio_rekurs: {  
  MAX_QUERIES_PER_HOUR count  
| MAX_UPDATES_PER_HOUR count  
| MAX_CONNECTIONS_PER_HOUR count  
| MAX_USER_CONNECTIONS count  
}
```

opcio_pwd (A partir de la versió 5.7): Opció que podem establir en la nostra clau de pas. Per exemple que expiri, que no expiri, que expiri dins d'un interval de dies, etc...

```
opcio_pwd: {  
  PASSWORD EXPIRE  
| PASSWORD EXPIRE DEFAULT  
| PASSWORD EXPIRE NEVER  
| PASSWORD EXPIRE INTERVAL <N> DAY  
}
```

Mirar la documentació de les polítiques d'expiració de paraules de pas.

<https://dev.mysql.com/doc/refman/8.0/en/password-expiration-policy.html>

Si utilitzem EXPIRE DEFAULT utilitzarem el temps de vida indicat al paràmetre de BD **default_password_lifetime=180** que el podem al fitxer de configuració my.cnf. Si deixem aquest paràmetre a 0 indicarem que el password mai caducarà.

Verificació del password (validate password)

<https://dev.mysql.com/doc/refman/8.0/en/validate-password.html>

A partir de la versió 5.7 de MySQL incorpora un pluguín (validate_password) que verifica i força a que tots els password compleixin una sèrie de regles:

- **validate_password_policy:** política de seguretat (LOW|MEDIUM(defecte)
| STRONG):

- LOW: Aquesta política només comprova la longitud el password i aquesta ha de ser de com a mínim 8 caràcters. Es pot canviar mitjançant *validate_password_length*.
- MEDIUM: Opció per defecte. Com a mínim ha de tenir 1 caràcter numèric, 1 caràcter en minúscules, 1 caràcter en majúscules i 1 caràcter especial (no alfanumèric). Tots aquests valors es poden canviar mitjançant *validate_password_number_count*, *validate_password_mixed_case_count*, *validate_password_special_char_count*.
- STRONG: Aquesta opció afegeix una condició en el password que és que els substrings de longitud 4 o més gran no coincideixin amb cap diccionari. El diccionari es pot especificar mitjançant *validate_password_dictionary_file*

Per canviar momentàniament la política del password es pot utilitzar la comanda `SET GLOBAL validate_password_policy`.

```
# Canviem la política de password
mysql> SET GLOBAL validate_password.policy=LOW;
```

```
# Canviem la longitud els password a 4 caràcters
mysql> SET GLOBAL validate_password.length=4;
```

III. Modificacions a usuaris

Ahora de realitzar canvis en l'autenticació d'un usuari ho podem fer de dues maneres.

Sintaxi

```
Opció 1
ALTER USER [IF EXISTS]
    <nom_usuari> [ opcio_auth],
    <nom_usuari> [opcio_auth]
[REQUIRE {NONE | opcio_tls }]
[WITH opcio_rekurs [opcio_rekurs] ...]
[opcio_pwd];

Opció 2
ALTER USER [IF EXISTS]
    <nom_usuari> IDENTIFIED BY 'clau_en_clar'
```

Les opcions de *opcio_auth*, *opcio_rekurs* i *opcio_pwd* són les mateixes que en la sentència de CREATE USER.

Exemple: Si volem modificar el *password* de l'usuari **perepi** i canviar a la vegada el número màxim de connexions d'aquest usuari i d'en **joanpau**.

```
ALTER USER
    'perepi'@'localhost' IDENTIFIED BY 'new_password',
    'joanpau'@'localhost'
    WITH MAX_USER_CONNECTIONS 2;
```

Nota: L'exemple anterior no modifica el password d'en **joanpau**

També podem renombar el nom d'un usuari mitjançant la sentència RENAME USER

Sintaxi:

```
RENAME USER <usuari_vell> TO <nou_usuari>
    [,<usuari_vell> TO <nou_usuari>]...
```

Exemple:

```
RENAME USER 'perepi'@'localhost' TO 'perepi'@'127.0.0.1';
```

També podem canviar la paraula de pas d'un usuari mitjançant la sentència SET PASSWORD.

Nota: En la documentació oficial es comenta que és millor utilitzar ALTER USER.

Sintaxi

```
SET PASSWORD [FOR <usuari>] = {  
    PASSWORD('clau en clar')  
    | 'clau_hash'  
};  
  
ALTER USER <usuari> IDENTIFIED BY 'clau en clar';
```

IV. Esborrat d'usuaris

Podem esborrar un usuari i els seus permisos associats mitjançant la sentència DROP USER.

Sintaxi:

```
DROP USER [IF EXISTS] <usuari> [, <usuari>] ...
```

Atenció: Això també esborra tots els permisos associats a aquest usuari.

V. Visualitzar el comptes creats

Tots els comptes de la BD es creen a la BD **mysql** dins la taula **user**. Per això per comprovar si un compte existeix podem realitzar la següent sentència SQL:

```
SELECT *  
  FROM mysql.user  
 WHERE user = 'perepi' AND host = '127.0.0.1';
```

VI. BLOQUEIG DE COMPTES

<https://dev.mysql.com/doc/refman/8.0/en/account-locking.html>

MySQL permet un mecanisme per bloquejar comptes mitjançant les sentències ACCOUNT LOCK i ACCOUNT UNLOCK.

1.11.2. Rols (a partir de la versió 8.0)

<https://dev.mysql.com/doc/refman/8.0/en/roles.htm>

<https://dev.mysql.com/doc/refman/8.0/en/role-names.html>

Un rol és un conjunt de privilegis identificats per un nom. De la mateixa manera que a un compte li assignem o traiem permisos, en un rol podem fer el mateix.

Un rol es pot assignar a un o més usuaris.

Un dels objectius que volem utilitzar amb els rols és la independència dels permisos i els comptes d'usuaris. Vindria a tenir un comportament similar als grups dels sistemes operatius.

Per exemple: Imaginem que tenim una bd de *rrhh*.

Podem tenir 3 rols: *rrhh_developer*, *rrhh_read*, *rrhh_write* un per cada funció i llavors assignar aquests rols en els usuaris que volem que siguin desenvolupadors, puguin escriure o aquells que només puguin llegir de la bd.

Sintaxi:

```
CREATE ROLE [IF NOT EXISTS] role [, role ] ...
```

```
DROP ROLE [IF EXISTS] role [, role ] ...
```

Per poder crear rols cal tenir els permisos de `CREATE ROLE` o `CREATE USER`.
Per poder esborrar rols cal tenir els permisos de `DROP ROLE` o `CREATE USER`.

Els rols no tenen la part de host com tenen els comptes d'usuari.

Els rols venen desactivats per defecte. Per activar un rol a un compte d'usuari ho podem fer quan creem l'usuari `CREATE/ALTER USER` o bé mitjançant `SET DEFAULT ROLE`

```
SET DEFAULT ROLE  
  {NONE | ALL | role [, role ] ...}  
  TO user [, user ] ...
```

`NONE` indicarem que no li estem assignant cap rol al compte d'usuari.

`ALL`: indicarem que estem assignant tots els rols al compte d'usuari.

Però abans d'activar el rol s'ha d'assignar el rol al compte d'usuari mitjançant `GRANT` (Veure l'apartat d'assignació de permisos)

Exemple complet de creació i assignació de rols:

Creem els rols de lectura i escriptura sobre la base de dades rrhh.

```
CREATE ROLE 'rrhh_read', 'rrhh_write';
```

Afegim els permisos pertinents en els rols. *Veure l'apartat d'assignació de permisos.*

```
GRANT SELECT ON rrhh.* TO 'rrhh_read';  
GRANT ALL ON rrhh.* TO 'rrhh_write';
```

Si intentem d'activar el rol a un compte d'usuari sense assignar-lo primer ens donarà un error.

```
SET DEFAULT ROLE rrhh_read TO 'alumne'@'%';  
ERROR 3530 (HY000): `rrhh_read`@`%` is not granted to `alumne`@`%`
```

Asignem el rol rrhh_read al compte alumne@%. Això no significa que el tingui actiu.

```
GRANT 'rrhh_read' TO 'alumne'@'%';
```

Quan assignem un rol a un usuari no vol dir que estigui actiu. Si seguim l'exemple i intentem d'entrar a la BBDD rrhh veurem que no tenim permisos encara.

```
mysql> USE rrhh;  
ERROR 1044 (42000): Access denied for user 'alumne'@'%' to database  
'rrhh'
```

Per veure quins rols tenim actius podem utilitzar la funció **CURRENT_ROLE()**.

```
-- Si mirem quins rols tenim actius veurem que no n'hi ha cap.  
SELECT CURRENT_ROLE();  
+-----+  
| CURRENT_ROLE() |  
+-----+  
| NONE           |  
+-----+
```

Per poder activar el rol podem fer un SET DEFAULT ROLE (sessió) o bé fer un ALTER USER.

```
SET DEFAULT ROLE rrhh_read TO 'alumne'@'%';  
  
SELECT CURRENT_ROLE();  
  
+-----+
```

```

| CURRENT_ROLE() |
+-----+
| NONE           |
+-----+

-- Amb la comanda anterior hem activat el rol per defecte, però no
per aquesta sessió, sinó per les properes quan ens connectem.

-- Per activar el rol a la sessió actual cal utilitzar SET ROLE
SET ROLE rrhh_read;
-- Ara si, quan mirem quins rols tenim actius veiem que tenim
-- rrhh_read.

SELECT CURRENT_ROLE();

+-----+
| CURRENT_ROLE() |
+-----+
| rrhh_read      |
+-----+

```

Podem assignar rols a un usuari mitjançant la sentència SET ROL. Amb aquest sentència modifiquem els rols de la sessió actual. En aquest cas s'assignaran aquells rols que s'assignin de forma explícita i aquells que estiguin a dins de la variable de sistema [mandatory roles](#)

Exemples:

```

SET ROLE DEFAULT; -- Assigna els rols que té l'usuari especificats
quan el vàrem crear
SET ROLE 'role1', 'role2'; -- Assigna els rols role1 i role2 a la
sessió.
SET ROLE ALL;
SET ROLE ALL EXCEPT 'role1', 'role2';

```

Compte en no confondre aquestes dues sentències: SET DEFAULT ROLE i SET ROLE DEFAULT

- SET DEFAULT ROLE: Defineix quins rols del compte d'usuari s'han d'activar de manera predeterminada quan aquest es connecti.
- SET ROLE DEFAULT: Estableix els rols actius de la sessió actual com a rols predeterminats al compte d'usuari actual.

1.11.3. Gestió de permisos

Abans de saber com gestionar els permisos ens cal saber de quins permisos disposem en el SGBD MySQL

<https://dev.mysql.com/doc/refman/8.0/en/grant.html#grant-privileges>

<https://dev.mysql.com/doc/refman/8.0/en/privileges-provided.html>

Diferències entre GRANT del SQL estàndard amb el del MySQL:

<https://dev.mysql.com/doc/refman/8.0/en/grant.html#grant-mysql-vs-standard-sql>

MySQL proporciona privilegis dins de diferents contextos i diferents nivells:

- **Permisos d'administració (*administrative privileges*)**: Són els que s'utilitzen per operacions de gestió del servidor MySQL. Aquests permisos **són globals** perquè no afecten a cap base de dades en particular.
Per exemple: `CREATE USER, SHOW DATABASES, SHUTDOWN, ALL [PRIVILEGES]`.
- **Permisos a base de dades (*database privileges*)**: Aquest permisos s'apliquen a una base de dades i a tots els objectes que hi ha dins. Aquests permisos es poden aplicar a **una base de dades o globalment** (afectarà a totes les base de dades)
Per exemple: `CREATE, DROP, GRANT OPTION, REFERENCES`
- **Permisos a objectes (*objects privileges*)**: S'apliquen a **objectes concrets** de la BD com taules, índexs, vistes, procediments emmagatzemats, etc....
Per exemple: `ALTER, DELETE, INDEX, INSERT, SELECT, UPDATE, ...`

Els privilegis es guarden a diferents taules de concessió de permisos de la base de dades de sistema (metadades) `mysql`. (<https://dev.mysql.com/doc/refman/8.0/en/grant-tables.html>)

Aquesta taula es contemplen els permisos estàtics més comuns.

Permís	Context
CREATE	databases, tables, or indexes
DROP	databases, tables, or views
GRANT OPTION	databases, tables, or stored routines
REFERENCES	databases, tables
ALTER	tables
DELETE	tables
INDEX	tables
INSERT	tables or columns
SELECT	tables or columns
UPDATE	tables or columns
CREATE TEMPORARY TABLES	tables
TRIGGER	tables
CREATE VIEW	views
CREATE ROUTINE	stored routines
EXECUTE	stored routines
CREATE USER	server administration
SHOW DATABASES	server administration
SHUTDOWN	server administration
ALL [PRIVILEGES]	server administration

Per tal d'assignar o treure permisos utilitzarem les sentència GRANT o REVOKE respectivament.

I. Assignar permisos - Sintaxi GRANT

```
GRANT <tipus_permis> [(<llista_columnnes>)]  
    [,<tipus_permis> [(<llista_columnnes>)] ...  
    ON [{TABLE|FUNCTION|PROCEDURE}] <objecte>  
    TO <usuari> [, <usuari>]...  
    [WITH GRANT OPTION ];
```

On:

- **tipus_permis:** qualsevol permís del llistat anterior. Mirar documentació oficial de MySQL.
- **llistat_columnnes:** llistem sobre quines columnnes específiques hi estem aplicant el permís.
- **objecte:** Indiquem sobre quin/s objecte/s volem donar permisos. Per defecte t'utilitza TABLE com a tipus d'objecte.
 - { *
 - | *.*
 - | db_name.*
 - | db_name.tbl_name
 - | tbl_name
 - | db_name.routine_name
- **usuari:** Indiquem sobre quin/s usuari/s donarem aquest permís.

Exemple: Volem donar permís de SELECT sobre totes les taules de la BD rrhh a l'usuari perepi quan es connecti de qualsevol màquina. A més que pugui atorgar aquest permís a altres usuaris o rols.

```
GRANT SELECT ON rrhh.* TO 'perepi'@'%' WITH GRANT OPTION;  
  
-- Donem els permís al rol rrhh_read  
GRANT SELECT ON rrhh.* TO 'rrhh_read' WITH GRANT OPTION;
```

Exemple: Volem donar permís de SELECT i INSERT sobre totes les taules de la BD rrhh a l'usuari perepi quan es connecti des de la màquina 192.168.0.1.

```
GRANT SELECT,INSERT ON rrhh.* TO 'perepi'@'192.168.0.1';
```

Exemple: Volem donar tots els permisos d'accés a la taula clients de la BD botiga per l'usuari perepi quan es connecti des de qualsevol màquina.

```
GRANT ALL ON botiga.clients TO 'perepi'@'%';
```

Exemple: Donar permís a l'usuari perepi perquè pugui crear subrutines emmagatzemades dins de la BD botiga.

```
GRANT CREATE ROUTINE ON botiga.* TO 'perepi'@'%';
```

Exemple: Donar permís a l'usuari perepi perquè pugui executar la rutina sp_ActualitzarSaldos de la BD botiga.

```
GRANT EXECUTE ON PROCEDURE botiga.sp_ActualitzarSaldos TO 'perepi'@'%';
```

Exemple: Donar permís a l'usuari perepi perquè pugui crear taules a la base de dades de botiga.

```
GRANT CREATE ON botiga.* TO 'perepi'@'%';
```

Per tal d'assignar i treure permisos (GRANT/REVOKE) cal que el compte tingui el permís de GRANT OPTION.

Per assignar aquest permís cal afegir WITH GRANT OPTION a la sentència de GRANT.

Exemple: Volem que l'usuari perepi connectat des de qualsevol màquina tingui tots els permisos sobre la base de dades botiga incloent el permís de gestió de permisos.

```
GRANT ALL ON botiga.* TO 'perepi'@%' WITH GRANT OPTION;
```

II. Assignar permisos - Sintaxi GRANT role

Assignar un/ o varis rol/s en un o diferent usuari/s.

```
GRANT role [, role]... TO user_or_role [, user_or_role] ....  
[WITH GRANT OPTION];
```

On:

- **role:** Nom del rol que volem assignar documentació oficial de MySQL.
- **user_or_role:** usuari o rol a qui va dirigit el rol. Com es pot observar podem establir una estructura jeràrquica de rols.
- **WITH GRANT OPTION:** Indiquem que l'usuari o el rol podrà donar també aquest rol.

III. Treure permisos - Sintaxi REVOKE

Tal i com hem dit per sostreure permisos als usuaris utilitzarem la sentència REVOKE.

<http://dev.mysql.com/doc/refman/8.0/en/revoke.html>

```

REVOKE <tipus_permis> [(<llista_columnnes>)]
[,<tipus_permis> [(<llista_columnnes>)]] ...
ON [{TABLE|FUNCTION|PROCEDURE}] <objecte>
FROM <usuari> [, <usuari>]...;

REVOKE ALL PRIVILEGES, GRANT OPTION
FROM <usuari> [, <usuari>]...;

```

On:

- **tipus_permis:** qualsevol permís del llistat anterior. Mirar documentació oficial de MySQL.
- **llistat_columnnes:** llistem sobre quines columnnes específiques hi estem aplicant el permís.
- **objecte:** Indiquem sobre quin/s objecte/s volem donar permisos.


```

      {
        *
      | *. *
      | db_name.*
      | db_name.tbl_name
      | tbl_name
      | db_name.routine_name
      }

```
- **usuari:** Indiquem sobre quin/s usuari/s li sostraiem el permís. Si només especifiquem la part de l'usuari s'utilitzarà '%' en la part de hostname

Exemple: Volem que l'usuari perepi no pugui esborrar registres de la taula clients de la BD botiga.

```

REVOKE DELETE ON botiga.clients FROM 'perepi'@'%';

REVOKE DELETE ON botiga.clients FROM 'perepi';

```

Exemple: Si volem sostreure tots els permisos d'un usuari:

```

REVOKE ALL PRIVILEGES, GRANT OPTION FROM perepi;

```

IV. Treure permisos - Sintaxi REVOKE

De la mateixa manera que fèiem amb l'assignació de permisos també podem treure rols als usuaris.

```

REVOKE role [, role].... FROM user_or_role [, user_or_role] ....;

```

V. Determinar permisos

Per determinar quins permisos té cada compte d'usuari podem utilitzar la sentència **SHOW GRANTS**.

<http://dev.mysql.com/doc/refman/8.0/en/show-grants.html>

Sintaxi

```
SHOW GRANTS [FOR <usuari>
             [USING <role> [,role]... ]]
```

Exemple: Si volem mostrar els permisos de l'usuari actual ho podem fer mitjançant les següents sentències.

```
SHOW GRANTS;
SHOW GRANTS FOR CURRENT_USER;
SHOW GRANTS FOR CURRENT_USER();
```

Exemple: Si volem saber quins permisos té l'usuari perepi quan es connecta des de la màquina local (localhost/màquina a on hi ha el server)

```
SHOW GRANTS FOR 'perepi'@'localhost';
```

Si no indiquem res a la part de host s'entendrà que volem saber des de qualsevol host.

```
SHOW GRANTS FOR 'perepi';
```

Exemple Si volem saber quins permisos té l'usuari perepi quan es connecta des de la màquina local amb un rol determinat (rrhh_write)

```
SHOW GRANTS FOR 'perepi'@'localhost' USING rrhh_write;

-- Cal que l'usuari tingui assignat aquest rol. Si no està assignat
ens donarà el següent error:
ERROR 3530 (HY000): 'rrhh_write' is not granted to
'perepi'@'localhost'
```

L'aplicació dels permisos no es fa de forma immediata i tenim una comanda/sentència per forçar a que els permisos s'apliquin, **FLUSH PRIVILEGES**

Moltes vegades i en moltes documentacions trobareu després d'un GRANT o un REVOKE la comanda de FLUSH PRIVILEGES.

Així doncs, quan hem d'utilitzar FLUSH PRIVILEGES?

La comanda de FLUSH PRIVILEGES existeix perquè MySQL manté a memòria de les taules dels permisos dels usuaris per no consultar-les cada vegada que un usuari realitza una instrucció. D'aquesta manera MySQL millora el rendiment.

L'objectiu d'aquesta comanda és precisament recarregar aquesta caché a memòria. Força a carregar els permisos de disc a memòria altre cop.

<https://dev.mysql.com/doc/refman/8.0/en/privilege-changes.html>

Si modifiquem els privilegis utilitzant les sentències de gestió com GRANT, REVOKE, SET PASSWORD, RENAME USER, ... No caldrà realitzar un FLUSH PRIVILEGES perquè MySQL ho farà per nosaltres.

1.11.4. Comprovació dels permisos de rol

Per poder verificar quins permisos té un usuari utilitzem la comanda SHOW GRANTS FOR <usuari>. Però en aquest cas només es mostren els permisos explícits i no els derivats dels rols.

S'ha dit en el punt anterior, però per saber quin permisos té l'usuari a través dels seus rols hem d'afegir la clàusula USING <role>

```
SHOW GRANTS FOR <usuari>
[USING role [, role] ...]]
```

<https://dev.mysql.com/doc/refman/8.0/en/roles.html#roles-activating>

1.11.5. Intercambiabilitat d'usuaris i rols

Es poden utilitzar rols o usuaris indistintament alhora d'assignar permisos.

Mirem aquest exemple. Assignem a l'usuari u2 i al rol r2 permisos u1 i r1 i tal i fa són rols o usuaris.

```
CREATE USER 'u1';
CREATE ROLE 'r1';

GRANT SELECT ON db1.* TO 'u1';
GRANT SELECT ON db2.* TO 'r1';

CREATE USER 'u2';
CREATE ROLE 'r2';

GRANT 'u1', 'r1' TO 'u2';
GRANT 'u1', 'r1' TO 'r2';
```